Trạng thái	Đã xong
Bắt đầu vào lúc	Thứ Bảy, 27 tháng 7 2024, 8:20 AM
Kết thúc lúc	Thứ Sáu, 23 tháng 8 2024, 4:27 PM
Thời gian thực hiện	27 Các ngày 8 giờ
Điểm	11,00/11,00
Điểm	**10,00** trên 10,00 (**100**%)

Câu hỏi **1**

Đúng

Đạt điểm 1,00 trên 1,00

Implement method bubbleSort() in class SLinkedList to sort this list in ascending order. After each bubble, we will print out a list to check (using printList).

```
#include <iostream>
#include <sstream>
using namespace std;

template <class T>
class SLinkedList {
public:
    class Node; // Forward declaration
protected:
    Node* head;
    Node* tail;
    int count;
public:
    SLinkedList()
    {
      this->head = nullptr;
      this->tail = nullptr;
      this->count = 0;
    }
    ~SLinkedList(){};
    void add(T e)
    {
        Node *pNew = new Node(e);

        if (this->count == 0)
        {
            this->head = this->tail = pNew;
        }
        else
        {
            this->tail->next = pNew;
            this->tail = pNew;
        }

        this->count++;
    }
    int size()
    {
        return this->count;
    }
    void printList()
    {
        stringstream ss;
        ss << "[";
        Node *ptr = head;
        while (ptr != tail)
        {
            ss << ptr->data << ",";
            ptr = ptr->next;
        }

        if (count > 0)
            ss << ptr->data << "]";
        else
            ss << "]";
        cout << ss.str() << endl;
    }
public:
    class Node {
    private:
        T data;
        Node* next;
        friend class SLinkedList<T>;
    public:
        Node() {
```

```
            next = 0;
        }
        Node(T data) {
            this->data = data;
            this->next = nullptr;
        }
    };

    void bubbleSort();
};
```

For example:

Test	Result

int arr[] = {9, 2, 8, 4, 1};
SLinkedList<int> list;
for(int i = 0; i <int(sizeof(arr))/4;i++)
list.add(arr[i]);
list.bubbleSort();
	[2,8,4,1,9]
	[2,4,1,8,9]
	[2,1,4,8,9]
	[1,2,4,8,9]

Answer: (penalty regime: 0 %)

Reset answer

```
template <class T>
void SLinkedList<T>::bubbleSort() {
    if (!head || !head->next) return; // If the list is empty or has only one element, it's already sorted

    bool swapped;
    Node *ptr1;
    Node *lptr = nullptr;

    do {
        swapped = false;
        ptr1 = head;

        while (ptr1->next != lptr) {
            if (ptr1->data > ptr1->next->data) {
                // Swap data
                T temp = ptr1->data;
                ptr1->data = ptr1->next->data;
                ptr1->next->data = temp;
                swapped = true;
            }
            ptr1 = ptr1->next;
        }
        lptr = ptr1;
        // Print the list after each pass
        if (swapped) { // Only print if there was a swap
            printList();
        }
    } while (swapped);
}
```

	Test	Expected	Got	

✓	int arr[] = {9, 2, 8, 4, 1};
	SLinkedList<int> list;
	for(int i = 0; i <int(sizeof(arr))/4;i++)
	list.add(arr[i]);
	list.bubbleSort();
		[2,8,4,1,9]
		[2,4,1,8,9]
		[2,1,4,8,9]
		[1,2,4,8,9]
			[2,8,4,1,9]
			[2,4,1,8,9]
			[2,1,4,8,9]
			[1,2,4,8,9]
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **2**

Đúng

Đạt điểm 1,00 trên 1,00

Implement static method selectionSort in class **Sorting** to sort an array in ascending order. After each selection, we will print out a list to check (using printArray).

```
#include <iostream>
using namespace std;

template <class T>
class Sorting
{
public:
    /* Function to print an array */
    static void printArray(T *start, T *end)
    {
        int size = end - start;
        for (int i = 0; i < size - 1; i++)
            cout << start[i] << ", ";
        cout << start[size - 1];
        cout << endl;
    }

    static void selectionSort(T *start, T *end);
};
```

For example:

Test	Result
int arr[] = {9, 2, 8, 1, 0, -2}; Sorting<int>::selectionSort(&arr[0], &arr[6]);	-2, 2, 8, 1, 0, 9 -2, 0, 8, 1, 2, 9 -2, 0, 1, 8, 2, 9 -2, 0, 1, 2, 8, 9 -2, 0, 1, 2, 8, 9

Answer: (penalty regime: 0 %)

Reset answer

```
template <class T>
void Sorting<T>::selectionSort(T *start, T *end)
{       int size = end -start;

        for (int i=0;i<size;i++){
            if (i==size-1) break;
            int min= i;
            for (int j=i+1;j<size;j++){
                if (start[j]<start[min]) min=j;
            }
            T tmp = start[i];
            start[i]= start[min];
            start[min]=tmp;

            for (int k=0;k<size;k++) {
                if (k==size-1) {
                    cout<< start[k]<<endl;
                    break;
        }
                cout<< start[k] <<", ";

```

```
}

        }

}

```

	Test	Expected	Got	
✓	int arr[] = {9, 2, 8, 1, 0, -2}; Sorting<int>::selectionSort(&arr[0], &arr[6]);	-2, 2, 8, 1, 0, 9 -2, 0, 8, 1, 2, 9 -2, 0, 1, 8, 2, 9 -2, 0, 1, 2, 8, 9 -2, 0, 1, 2, 8, 9	-2, 2, 8, 1, 0, 9 -2, 0, 8, 1, 2, 9 -2, 0, 1, 8, 2, 9 -2, 0, 1, 2, 8, 9 -2, 0, 1, 2, 8, 9	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **3**

Đúng

Đạt điểm 1,00 trên 1,00

Implement static methods **sortSegment** and **ShellSort** in class **Sorting** to sort an array in ascending order.

```
#ifndef SORTING_H
#define SORTING_H

#include <sstream>
#include <iostream>
#include <type_traits>
using namespace std;

template <class T>
class Sorting {
private:
    static void printArray(T* start, T* end)
    {
        int size = end - start;
        for (int i = 0; i < size; i++)
            cout << start[i] << " ";
        cout << endl;
    }

public:
    // TODO: Write your code here
    static void sortSegment(T* start, T* end, int segment_idx, int cur_segment_total);
    static void ShellSort(T* start, T* end, int* num_segment_list, int num_phases);
};

#endif /* SORTING_H */
```

For example:

Test	Result

`int num_segment_list[] = {1, 3, 5};`
`int num_phases = 3;`
`int array[] = { 10, 9, 8 , 7 , 6, 5, 4, 3, 2, 1 };`
 	
`Sorting<int>::ShellSort(&array[0], &array[10], &num_segment_list[0], num_phases);`
	`5 segments: 5 4 3 2 1 10 9 8 7 6`
	`3 segments: 2 1 3 5 4 7 6 8 10 9`
	`1 segments: 1 2 3 4 5 6 7 8 9 10`

Answer: (penalty regime: 0 %)

Reset answer

```
// TODO: Write your code here

static void sortSegment(T* start, T* end, int segment_idx, int cur_segment_total) {
    // TODO
    int curr=segment_idx+cur_segment_total;
    int count=end-start;
    while (curr<count) {
        int tmp=start[curr];
        int step=curr-cur_segment_total;
        while (step>=0 && tmp<start[step]) {
            start[step+cur_segment_total] =start[step];
            step-=cur_segment_total;
        }
        start[step+cur_segment_total] =tmp;
        curr+=cur_segment_total;
    }
}

```

```
19 static void ShellSort(T* start, T* end, int* num_segment_list, int num_phases) {
20     // TODO
21     // Note: You must print out the array after sorting segments to check whether your algorithm is true.
22     int k=num_segment_list[num_phases-1];
23     for (int i=num_phases-2; i>=-1; i--) {
24         int segment=0;
25         while (segment<=k) {
26             sortSegment(start,end,segment,k);
27             segment++;
28         }
29         cout<<k<<" segments: ";
30         printArray(start,end);
31         k=num_segment_list[i];
32     }
33 }
```

	Test	Expected	Got	
✓	int num_segment_list[] = {1, 3, 5}; int num_phases = 3; int array[] = { 10, 9, 8 , 7 , 6, 5, 4, 3, 2, 1 }; Sorting<int>::ShellSort(&array[0], &array[10], &num_segment_list[0], num_phases);	5 segments: 5 4 3 2 1 10 9 8 7 6 3 segments: 2 1 3 5 4 7 6 8 10 9 1 segments: 1 2 3 4 5 6 7 8 9 10	5 segments: 5 4 3 2 1 10 9 8 7 6 3 segments: 2 1 3 5 4 7 6 8 10 9 1 segments: 1 2 3 4 5 6 7 8 9 10	✓
✓	int num_segment_list[] = { 1, 2, 6 }; int num_phases = 3; int array[] = { 10, 9, 8 , 7 , 6, 5, 4, 3, 2, 1 }; Sorting<int>::ShellSort(&array[0], &array[10], &num_segment_list[0], num_phases);	6 segments: 4 3 2 1 6 5 10 9 8 7 2 segments: 2 1 4 3 6 5 8 7 10 9 1 segments: 1 2 3 4 5 6 7 8 9 10	6 segments: 4 3 2 1 6 5 10 9 8 7 2 segments: 2 1 4 3 6 5 8 7 10 9 1 segments: 1 2 3 4 5 6 7 8 9 10	✓
✓	int num_segment_list[] = { 1, 2, 5 }; int num_phases = 3; int array[] = { 10, 9, 8 , 7 , 6, 5, 4, 3, 2, 1 }; Sorting<int>::ShellSort(&array[0], &array[10], &num_segment_list[0], num_phases);	5 segments: 5 4 3 2 1 10 9 8 7 6 2 segments: 1 2 3 4 5 6 7 8 9 10 1 segments: 1 2 3 4 5 6 7 8 9 10	5 segments: 5 4 3 2 1 10 9 8 7 6 2 segments: 1 2 3 4 5 6 7 8 9 10 1 segments: 1 2 3 4 5 6 7 8 9 10	✓
✓	int num_segment_list[] = { 1, 2, 3 }; int num_phases = 3; int array[] = { 10, 9, 8 , 7 , 6, 5, 4, 3, 2, 1 }; Sorting<int>::ShellSort(&array[0], &array[10], &num_segment_list[0], num_phases);	3 segments: 1 3 2 4 6 5 7 9 8 10 2 segments: 1 3 2 4 6 5 7 9 8 10 1 segments: 1 2 3 4 5 6 7 8 9 10	3 segments: 1 3 2 4 6 5 7 9 8 10 2 segments: 1 3 2 4 6 5 7 9 8 10 1 segments: 1 2 3 4 5 6 7 8 9 10	✓
✓	int num_segment_list[] = { 1, 5, 8, 10 }; int num_phases = 4; int array[] = { 3, 5, 7, 10 ,12, 14, 15, 13, 1, 2, 9, 6, 4, 8, 11 }; Sorting<int>::ShellSort(&array[0], &array[15], &num_segment_list[0], num_phases);	10 segments: 3 5 4 8 11 14 15 13 1 2 9 6 7 10 12 8 segments: 1 2 4 6 7 10 12 13 3 5 9 8 11 14 15 5 segments: 1 2 4 3 5 9 8 11 6 7 10 12 13 14 15 1 segments: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15	10 segments: 3 5 4 8 11 14 15 13 1 2 9 6 7 10 12 8 segments: 1 2 4 6 7 10 12 13 3 5 9 8 11 14 15 5 segments: 1 2 4 3 5 9 8 11 6 7 10 12 13 14 15 1 segments: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **4**

Đúng

Đạt điểm 1,00 trên 1,00

Implement static methods **Partition** and **QuickSort** in class Sorting to sort an array in **DESCENDING order**.

```
#ifndef SORTING_H
#define SORTING_H
#include <sstream>
#include <iostream>
#include <type_traits>
using namespace std;
template <class T>
class Sorting {
private:
    static T* Partition(T* start, T* end) ;
public:
    static void QuickSort(T* start, T* end) ;
};
#endif /* SORTING_H */
```

For example:

Test	Result
int array[] = { 3, 5, 7, 10 ,12, 14, 15, 13, 1, 2, 9, 6, 4, 8, 11, 16, 17, 18, 20, 19 }; cout << "Index of pivots: "; Sorting<int>::QuickSort(&array[0], &array[20]); cout << "\n"; cout << "Array after sorting: "; for (int i : array) cout << i << " ";	Index of pivots: 17 2 1 0 7 6 1 0 1 0 0 0 3 0 1 0 0 0 0 0 Array after sorting: 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1

Answer: (penalty regime: 0 %)

Reset answer

```
1  static T* Partition(T* start, T* end) {
2      // TODO: return the pointer which points to the pivot after rearrange the array.
3      T* i = start;
4      T* j = end;
5      do {
6          do {
7              i++;
8          } while (*i > *start);
9          while (*j < *start) {
10             j--;
11         }
12         swap(*i, *j);
13     } while (i < j);
14     swap(*i, *j);
15     swap(*j, *start);
16     return j;
17 }
18
19 static void QuickSort(T* start, T* end, T* begin= nullptr) {
20     // TODO
21     // In this question, you must print out the index of pivot in subarray after everytime calling method Par
22
23     if (start <= end) {
24
25         T* pivotIndex = Partition(start, end);
26         if(!begin) begin = end;
27         if(start!=begin) cout <<  pivotIndex - start << " ";
```

```
28          QuickSort(start, pivotIndex-1, begin);
29          QuickSort(pivotIndex+1, end, begin);
30
31      }
32
33  }
```

	Test	Expected	Got	
✓	int array[] = { 3, 5, 7, 10 ,12, 14, 15, 13, 1, 2, 9, 6, 4, 8, 11, 16, 17, 18, 20, 19 }; cout << "Index of pivots: "; Sorting<int>::QuickSort(&array[0], &array[20]); cout << "\n"; cout << "Array after sorting: "; for (int i : array) cout << i << " ";	Index of pivots: 17 2 1 0 7 6 1 0 1 0 0 0 3 0 1 0 0 0 0 0 Array after sorting: 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1	Index of pivots: 17 2 1 0 7 6 1 0 1 0 0 0 3 0 1 0 0 0 0 0 Array after sorting: 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **5**

Đúng

Đạt điểm 1,00 trên 1,00

Implement static methods **Merge** and **MergeSort** in class Sorting to sort an array in ascending order. The Merge method has already been defined a call to method printArray so you do not have to call this method again to print your array.

```
#ifndef SORTING_H
#define SORTING_H
#include <iostream>
using namespace std;
template <class T>
class Sorting {
public:
    /* Function to print an array */
    static void printArray(T *start, T *end)
    {
        long size = end - start + 1;
        for (int i = 0; i < size - 1; i++)
            cout << start[i] << ", ";
        cout << start[size - 1];
        cout << endl;
    }

    static void merge(T* left, T* middle, T* right){
        /*TODO*/
        Sorting::printArray(left, right);
    }
    static void mergeSort(T* start, T* end) {
       /*TODO*/
    }
};
#endif /* SORTING_H */
```

For example:

Test	Result
int arr[] = {0,2,4,3,1,4}; Sorting<int>::mergeSort(&arr[0], &arr[5]);	0, 2 0, 2, 4 1, 3 1, 3, 4 0, 1, 2, 3, 4, 4
int arr[] = {1}; Sorting<int>::mergeSort(&arr[0], &arr[0]);	

Answer: (penalty regime: 0, 0, 0, 5, 10, 15, ... %)

Reset answer

```
static void merge(T* left, T* middle, T* right) {
        int n1 = middle - left + 1;
        int n2 = right - middle;

        // Create temporary arrays
        T* L = new T[n1];
        T* R = new T[n2];

        // Copy data to temporary arrays L[] and R[]
        for (int i = 0; i < n1; i++)
            L[i] = left[i];
        for (int j = 0; j < n2; j++)
            R[j] = middle[1 + j];
```

```
        R[j] = middle[i + j];

        // Merge the temporary arrays back into the original array
        int i = 0; // Initial index of first subarray
        int j = 0; // Initial index of second subarray
        int k = 0; // Initial index of merged subarray
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) {
                left[k] = L[i];
                i++;
            } else {
                left[k] = R[j];
                j++;
            }
            k++;
        }

        // Copy the remaining elements of L[], if there are any
        while (i < n1) {
            left[k] = L[i];
            i++;
            k++;
        }

        // Copy the remaining elements of R[], if there are any
        while (j < n2) {
            left[k] = R[j];
            j++;
            k++;
        }

        // Clean up
        delete[] L;
        delete[] R;

        // Print the array after merging
        printArray(left, right);
    }

    static void mergeSort(T* start, T* end) {
        if (start < end) {
            T* middle = start + (end - start) / 2;

            // Sort first and second halves
            mergeSort(start, middle);
            mergeSort(middle + 1, end);

            // Merge the sorted halves
            merge(start, middle, end);
        }
    }
```

	Test	Expected	Got	
✓	int arr[] = {0,2,4,3,1,4}; Sorting<int>::mergeSort(&arr[0], &arr[5]);	0, 2 0, 2, 4 1, 3 1, 3, 4 0, 1, 2, 3, 4, 4	0, 2 0, 2, 4 1, 3 1, 3, 4 0, 1, 2, 3, 4, 4	✓
✓	int arr[] = {1}; Sorting<int>::mergeSort(&arr[0], &arr[0]);			✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **6**

Đúng

Đạt điểm 1,00 trên 1,00

The best way to sort a singly linked list given the head pointer is probably using merge sort.

Both Merge sort and Insertion sort can be used for linked lists. The slow random-access performance of a linked list makes other algorithms (such as quick sort) perform poorly, and others (such as heap sort) completely impossible. Since worst case time complexity of Merge Sort is O(nLogn) and Insertion sort is O(n^2), merge sort is preferred.

Additionally, Merge Sort for linked list only requires a small constant amount of auxiliary storage.

To gain a deeper understanding about Merge sort on linked lists, let's implement **mergeLists** and **mergeSortList** function below

Constraints:

0 <= list.length <= 10^4
0 <= node.val <= 10^6

Use the nodes in the original list and don't modify ListNode's val attribute.

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int _val = 0, ListNode* _next = nullptr) : val(_val), next(_next) { }
};

// Merge two sorted lists
ListNode* mergeSortList(ListNode* head);

// Sort an unsorted list given its head pointer
ListNode* mergeSortList(ListNode* head);
```

For example:

Test	Input	Result

`int arr1[] = {1, 3, 5, 7, 9};`
`int arr2[] = {2, 4, 6, 8};`
`unordered_map<ListNode*, int> nodeAddr;`
`ListNode* a = init(arr1, sizeof(arr1) / 4, nodeAddr);`
`ListNode* b = init(arr2, sizeof(arr2) / 4, nodeAddr);`
`ListNode* merged = mergeLists(a, b);`
`try {`
`  printList(merged, nodeAddr);`
`}`
`catch(char const* err) {`
`  cout << err << '\n';`
`}`
`freeMem(merged);`
		1 2 3 4 5 6 7 8 9

Test	Input	Result

int size;
cin >> size;
int* array = new int[size];
for(int i = 0; i < size; i++) cin >> array[i];
unordered_map<ListNode*, int> nodeAddr;
ListNode* head = init(array, size, nodeAddr);
ListNode* sorted = mergeSortList(head);
try {
printList(sorted, nodeAddr);
}
catch(char const* err) {
cout << err << '\n';
}
freeMem(sorted);
delete[] array;
	9
	9 3 8 2 1 6 7 4 5
		1 2 3 4 5 6 7 8 9

Answer: (penalty regime: 0 %)

Reset answer

```
// Function to merge two sorted lists
ListNode* mergeLists(ListNode* l1, ListNode* l2) {
    ListNode dummy;
    ListNode* tail = &dummy;
    dummy.next = nullptr;

    while (l1 != nullptr && l2 != nullptr) {
        if (l1->val <= l2->val) {
            tail->next = l1;
            l1 = l1->next;
        } else {
            tail->next = l2;
            l2 = l2->next;
        }
        tail = tail->next;
    }
    tail->next = (l1 != nullptr) ? l1 : l2;

    return dummy.next;
}

// Function to split the list into two halves
ListNode* splitList(ListNode* head) {
    if (head == nullptr || head->next == nullptr) return head;

    ListNode* slow = head;
    ListNode* fast = head;
    ListNode* prev = nullptr;

    while (fast != nullptr && fast->next != nullptr) {
        prev = slow;
        slow = slow->next;
        fast = fast->next->next;
    }

    if (prev != nullptr) prev->next = nullptr;

    return slow;
}

// Function to sort the list using merge sort
ListNode* mergeSortList(ListNode* head) {
    if (head == nullptr || head->next == nullptr) return head;

    ListNode* mid = splitList(head);

    ListNode* left = mergeSortList(head);
```

```
    ListNode* right = mergeSortList(mid);

    return mergeLists(left, right);
}
```

	Test	Input	Expected	Got	

✓	`int arr1[] = {1, 3, 5, 7, 9};`
	`int arr2[] = {2, 4, 6, 8};`
	`unordered_map<ListNode*, int> nodeAddr;`
	`ListNode* a = init(arr1, sizeof(arr1) / 4, nodeAddr);`
	`ListNode* b = init(arr2, sizeof(arr2) / 4, nodeAddr);`
	`ListNode* merged = mergeLists(a, b);`
	`try {`
	`printList(merged, nodeAddr);`
	`}`
	`catch(char const* err) {`
	`cout << err << '\n';`
	`}`
	`freeMem(merged);`
			1 2 3 4 5 6 7 8 9	1 2 3 4 5 6 7 8 9	✓

✓	`int size;`
	`cin >> size;`
	`int* array = new int[size];`
	`for(int i = 0; i < size; i++) cin >> array[i];`
	`unordered_map<ListNode*, int> nodeAddr;`
	`ListNode* head = init(array, size, nodeAddr);`
	`ListNode* sorted = mergeSortList(head);`
	`try {`
	`printList(sorted, nodeAddr);`
	`}`
	`catch(char const* err) {`
	`cout << err << '\n';`
	`}`
	`freeMem(sorted);`
	`delete[] array;`
		9
		9 3 8 2 1 6 7 4 5
			1 2 3 4 5 6 7 8 9	1 2 3 4 5 6 7 8 9	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **7**

Đúng

Đạt điểm 1,00 trên 1,00

Implement static methods **merge, InsertionSort** and **TimSort** in class **Sorting** to sort an array in ascending order.

merge is responsible for merging two sorted subarrays. It takes three pointers: start, middle, and end, representing the left, middle, and right portions of an array.

InsertionSort is an implementation of the insertion sort algorithm. It takes two pointers, start and end, and sorts the elements in the range between them in ascending order using the insertion sort technique.

TimSort is an implementation of the TimSort algorithm, a hybrid sorting algorithm that combines insertion sort and merge sort. It takes two pointers, start and end, and an integer min_size, which determines the minimum size of subarrays to be sorted using insertion sort. The function first applies insertion sort to small subarrays, prints the intermediate result, and then performs merge operations to combine sorted subarrays until the entire array is sorted.

```
#ifndef SORTING_H
#define SORTING_H
#include <sstream>
#include <iostream>
#include <type_traits>
using namespace std;
template <class T>
class Sorting {
private:
    static void printArray(T* start, T* end)
    {
        int  size = end - start;
        for (int i = 0; i < size - 1; i++)
            cout << start[i] << " ";
        cout << start[size - 1];
        cout << endl;
    }

    static void merge(T* start, T* middle, T* end) ;
public:
    static void InsertionSort(T* start, T* end) ;
    static void TimSort(T* start, T* end, int min_size) ;
};
#endif /* SORTING_H */
```

For example:

Test	Result
int array[] = { 19, 20, 18, 17 ,12, 13, 14, 15, 1, 2, 9, 6, 4, 7, 11, 16, 10, 8, 5, 3 }; int min_size = 4; Sorting<int>::TimSort(&array[0], &array[20], min_size);	Insertion Sort: 17 18 19 20 12 13 14 15 1 2 6 9 4 7 11 16 3 5 8 10 Merge 1: 12 13 14 15 17 18 19 20 1 2 6 9 4 7 11 16 3 5 8 10 Merge 2: 12 13 14 15 17 18 19 20 1 2 4 6 7 9 11 16 3 5 8 10 Merge 3: 12 13 14 15 17 18 19 20 1 2 4 6 7 9 11 16 3 5 8 10 Merge 4: 1 2 4 6 7 9 11 12 13 14 15 16 17 18 19 20 3 5 8 10 Merge 5: 1 2 4 6 7 9 11 12 13 14 15 16 17 18 19 20 3 5 8 10 Merge 6: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
int array[] = { 3, 20, 18, 17 ,12, 13, 14, 15, 1, 2, 9, 6, 4, 7, 11, 16, 10, 8, 5, 19 }; int min_size = 4; Sorting<int>::TimSort(&array[0], &array[20], min_size);	Insertion Sort: 3 17 18 20 12 13 14 15 1 2 6 9 4 7 11 16 5 8 10 19 Merge 1: 3 12 13 14 15 17 18 20 1 2 6 9 4 7 11 16 5 8 10 19 Merge 2: 3 12 13 14 15 17 18 20 1 2 4 6 7 9 11 16 5 8 10 19 Merge 3: 3 12 13 14 15 17 18 20 1 2 4 6 7 9 11 16 5 8 10 19 Merge 4: 1 2 3 4 6 7 9 11 12 13 14 15 16 17 18 20 5 8 10 19 Merge 5: 1 2 3 4 6 7 9 11 12 13 14 15 16 17 18 20 5 8 10 19 Merge 6: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Answer: (penalty regime: 0 %)

Reset answer

```
static void merge(T* start, T* middle, T* end) {
    T* left = start;
    T* right = end;
    int n1 = middle - left + 1;
    int n2 = right - middle;

    T *L = new T[n1], *R = new T[n2];

    for (int i = 0; i < n1; i++)
        L[i] = left[i];
    for (int j = 0; j < n2; j++)
        R[j] = middle[1 + j];

    int i = 0, j = 0, k = 0;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            left[k] = L[i];
            i++;
        } else {
            left[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        left[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
```

```
    while (j < n2) {
        left[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}

static void InsertionSort(T* start, T* end) {
    for (int i = 0; i < end - start; i++) {
        T min = start[i];
        int id = i;
        for (int j = i; j < end - start; j++) {
            if (start[j] < min) {
                id = j;
                min = start[j];
            }
        }
        T temp = start[i];
        start[i] = start[id];
        start[id] = temp;
    }
}

static void TimSort(T* start, T* end, int min_size) {
    cout << "Insertion Sort: ";
    for (int i = 0; i < end - start; i += min_size) {
        if (start + i + min_size < end)
            InsertionSort(start + i, start + i + min_size);
        else
            InsertionSort(start + i, end);
    }
    printArray(start, end);

    int i = 1;
    for (int size = min_size; size < end - start; size = 2 * size) {
        for (int left = 0; left < end - start; left += 2 * size, i++) {
            cout << "Merge " << i << ": ";
            int mid = min<int>(left + size - 1, end - start - 1);
            int right = min<int>(left + 2 * size - 1, end - start - 1);
            merge(&start[left], &start[mid], &start[right]);
            printArray(start, end);
        }
    }
}
```

	Test	Expected	Got	
✓	int array[] = { 19, 20, 18, 17 ,12, 13, 14, 15, 1, 2, 9, 6, 4, 7, 11, 16, 10, 8, 5, 3 }; int min_size = 4; Sorting<int>::TimSort(&array[0], &array[20], min_size);	Insertion Sort: 17 18 19 20 12 13 14 15 1 2 6 9 4 7 11 16 3 5 8 10 Merge 1: 12 13 14 15 17 18 19 20 1 2 6 9 4 7 11 16 3 5 8 10 Merge 2: 12 13 14 15 17 18 19 20 1 2 4 6 7 9 11 16 3 5 8 10 Merge 3: 12 13 14 15 17 18 19 20 1 2 4 6 7 9 11 16 3 5 8 10 Merge 4: 1 2 4 6 7 9 11 12 13 14 15 16 17 18 19 20 3 5 8 10 Merge 5: 1 2 4 6 7 9 11 12 13 14 15 16 17 18 19 20 3 5 8 10 Merge 6: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20	Insertion Sort: 17 18 19 20 12 13 14 15 1 2 6 9 4 7 11 16 3 5 8 10 Merge 1: 12 13 14 15 17 18 19 20 1 2 6 9 4 7 11 16 3 5 8 10 Merge 2: 12 13 14 15 17 18 19 20 1 2 4 6 7 9 11 16 3 5 8 10 Merge 3: 12 13 14 15 17 18 19 20 1 2 4 6 7 9 11 16 3 5 8 10 Merge 4: 1 2 4 6 7 9 11 12 13 14 15 16 17 18 19 20 3 5 8 10 Merge 5: 1 2 4 6 7 9 11 12 13 14 15 16 17 18 19 20 3 5 8 10 Merge 6: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20	✓
✓	int array[] = { 3, 20, 18, 17 ,12, 13, 14, 15, 1, 2, 9, 6, 4, 7, 11, 16, 10, 8, 5, 19 }; int min_size = 4; Sorting<int>::TimSort(&array[0], &array[20], min_size);	Insertion Sort: 3 17 18 20 12 13 14 15 1 2 6 9 4 7 11 16 5 8 10 19 Merge 1: 3 12 13 14 15 17 18 20 1 2 6 9 4 7 11 16 5 8 10 19 Merge 2: 3 12 13 14 15 17 18 20 1 2 4 6 7 9 11 16 5 8 10 19 Merge 3: 3 12 13 14 15 17 18 20 1 2 4 6 7 9 11 16 5 8 10 19 Merge 4: 1 2 3 4 6 7 9 11 12 13 14 15 16 17 18 20 5 8 10 19 Merge 5: 1 2 3 4 6 7 9 11 12 13 14 15 16 17 18 20 5 8 10 19 Merge 6: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20	Insertion Sort: 3 17 18 20 12 13 14 15 1 2 6 9 4 7 11 16 5 8 10 19 Merge 1: 3 12 13 14 15 17 18 20 1 2 6 9 4 7 11 16 5 8 10 19 Merge 2: 3 12 13 14 15 17 18 20 1 2 4 6 7 9 11 16 5 8 10 19 Merge 3: 3 12 13 14 15 17 18 20 1 2 4 6 7 9 11 16 5 8 10 19 Merge 4: 1 2 3 4 6 7 9 11 12 13 14 15 16 17 18 20 5 8 10 19 Merge 5: 1 2 3 4 6 7 9 11 12 13 14 15 16 17 18 20 5 8 10 19 Merge 6: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **8**

Đúng

Đạt điểm 1,00 trên 1,00

A hotel has m rooms left, there are n people who want to stay in this hotel. You have to distribute the rooms so that as many people as possible will get a room to stay.
However, each person has a desired room size, he/she will accept the room if its size is close enough to the desired room size.
More specifically, if the maximum difference is k, and the desired room size is x, then he or she will accept a room if its size is between x - k and x + k
Determine the maximum number of people who will get a room to stay.

input:
vector<int> rooms: rooms[i] is the size of the ith room
vector<int> people: people[i] the desired room size of the ith person
int k: maximum allowed difference. If the desired room size is x, he or she will accept a room if its size is between x - k and x + k

output:
the maximum number of people who will get a room to stay.

Note: The iostream, vector and algorithm library are already included for you.

Constraints:
1 <= rooms.length, people.length <= 2 * 10^5
0 <= k <= 10^9
1 <= rooms[i], people[i] <= 10^9

Example 1:

Input:
rooms = {57, 45, 80, 65}
people = {30, 60, 75}
k = 5
Output:
2

Explanation:
2 is the maximum amount of people that can stay in this hotel.
There are 3 people and 4 rooms, the first person cannot stay in any room, the second and third person can stay in the first and third room, respectively

Example 2:

Input:
rooms = {59, 5, 65, 15, 42, 81, 58, 96, 50, 1}
people = {18, 59, 71, 65, 97, 83, 80, 68, 92, 67}
k = 1000
Output:
10

For example:

Test	Input	Result

int peopleCount, roomCount, k;
cin >> peopleCount >> roomCount >> k;
 	
vector<int> people(peopleCount);
vector<int> rooms(roomCount);
 	
for(int i = 0; i < peopleCount; i++)
cin >> people[i];
for(int i = 0; i < roomCount; i++)
cin >> rooms[i];
cout << maxNumberOfPeople(rooms, people, k) << '\n';
	3 4 5
	30 60 75
	57 45 80 65
		2

int peopleCount, roomCount, k;
cin >> peopleCount >> roomCount >> k;
 	
vector<int> people(peopleCount);
vector<int> rooms(roomCount);
 	
for(int i = 0; i < peopleCount; i++)
cin >> people[i];
for(int i = 0; i < roomCount; i++)
cin >> rooms[i];
cout << maxNumberOfPeople(rooms, people, k) << '\n';
	10 10 1000
	18 59 71 65 97 83 80 68 92 67
	59 5 65 15 42 81 58 96 50 1
		10

Answer: (penalty regime: 0 %)

Reset answer

```
int maxNumberOfPeople(std::vector<int>& rooms, std::vector<int>& people, int k) {
    std::sort(rooms.begin(), rooms.end());
    std::sort(people.begin(), people.end());

    int roomPointer = 0, personPointer = 0;
    int maxPeople = 0;

while (roomPointer < rooms.size() && personPointer < people.size()) {
if (rooms[roomPointer] >= people[personPointer]-k && rooms[roomPointer]<=people[personPointer]+k)
{            maxPeople++;
            roomPointer++;
            personPointer++;
        } else if (rooms[roomPointer] < people[personPointer] - k) {
            roomPointer++;
        } else {
            personPointer++;
        }
    }

    return maxPeople;
}
```

	Test	Input	Expected	Got	

✓	`int peopleCount, roomCount, k;`
	`cin >> peopleCount >> roomCount >> k;`
	
	`vector<int> people(peopleCount);`
	`vector<int> rooms(roomCount);`
	
	`for(int i = 0; i < peopleCount; i++)`
	`  cin >> people[i];`
	`for(int i = 0; i < roomCount; i++)`
	`  cin >> rooms[i];`
	`cout << maxNumberOfPeople(rooms, people, k) << '\n';`
		3 4 5
		30 60 75
		57 45 80 65
			2	2	✓

✓	`int peopleCount, roomCount, k;`
	`cin >> peopleCount >> roomCount >> k;`
	
	`vector<int> people(peopleCount);`
	`vector<int> rooms(roomCount);`
	
	`for(int i = 0; i < peopleCount; i++)`
	`  cin >> people[i];`
	`for(int i = 0; i < roomCount; i++)`
	`  cin >> rooms[i];`
	`cout << maxNumberOfPeople(rooms, people, k) << '\n';`
		10 10 1000
		18 59 71 65 97 83 80 68 92 67
		59 5 65 15 42 81 58 96 50 1
			10	10	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi 9

Đúng

Đạt điểm 1,00 trên 1,00

Given a list of distinct unsorted integers nums.

Your task is to implement a function with following prototype:

```
int minDiffPairs(int* arr, int n);
```

This function identify and return all pairs of elements with the smallest absolute difference among them. If there are multiple pairs that meet this criterion, the function should find and return all of them.

Note: Following libraries are included: iostream, string, algorithm, sstream

For example:

Test	Result
`int arr[] = {10, 5, 7, 9, 15, 6, 11, 8, 12, 2};` `cout << minDiffPairs(arr, 10);`	(5, 6), (6, 7), (7, 8), (8, 9), (9, 10), (10, 11), (11, 12)
`int arr[] = {10};` `cout << minDiffPairs(arr, 1);`	
`int arr[] = {10, -1, -150, 200};` `cout << minDiffPairs(arr, 4);`	(-1, 10)

Answer: (penalty regime: 0 %)

Reset answer

```
#include <climits>
string minDiffPairs(int* arr, int n) {
    if (n < 2) {
        return "";
    }

    // Sort the array
    sort(arr, arr + n);

    // Find the minimum difference
    int minDiff = INT_MAX;
    for (int i = 1; i < n; ++i) {
        minDiff = min(minDiff, arr[i] - arr[i - 1]);
    }

    // Find all pairs with the minimum difference
    ostringstream result;
    bool firstPair = true;
    for (int i = 1; i < n; ++i) {
        if (arr[i] - arr[i - 1] == minDiff) {
            if (!firstPair) {
                result << ", ";
            }
            result << "(" << arr[i - 1] << ", " << arr[i] << ")";
            firstPair = false;
        }
    }

    return result.str();
}
```

	Test	Expected	Got	
✓	`int arr[] = {10, 5, 7, 9, 15, 6, 11, 8, 12, 2};` `cout << minDiffPairs(arr, 10);`	`(5, 6), (6, 7), (7, 8), (8, 9), (9, 10), (10, 11), (11, 12)`	`(5, 6), (6, 7), (7, 8), (8, 9), (9, 10), (10, 11), (11, 12)`	✓
✓	`int arr[] = {10};` `cout << minDiffPairs(arr, 1);`			✓
✓	`int arr[] = {10, -1, -150, 200};` `cout << minDiffPairs(arr, 4);`	`(-1, 10)`	`(-1, 10)`	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **10**

Đúng

Đạt điểm 1,00 trên 1,00

Print the elements of an array in the decreasing frequency order while preserving the relative order of the elements.

Students are not allowed to use map/unordered map.

iostream, algorithm libraries are included.

For example:

Test	Result

`int arr[] = {-4,1,2,2,-4,9,1,-1};`
`int n = sizeof(arr) / sizeof(arr[0]);`
 	
`sortByFrequency(arr, n);`
 	
`for (int i = 0; i < n; i++)`
`  cout << arr[i] << " ";`
	-4 -4 1 1 2 2 9 -1

`int arr[] = {-5,3,8,1,-9,-9};`
`int n = sizeof(arr) / sizeof(arr[0]);`
 	
`sortByFrequency(arr, n);`
 	
`for (int i = 0; i < n; i++)`
`  cout << arr[i] << " ";`
	-9 -9 -5 3 8 1

Answer: (penalty regime: 0 %)

Reset answer

```
struct Element {
    int value;
    int frequency;
    int firstIndex;
};

// Comparator function to sort based on frequency and first occurrence
bool compare(Element a, Element b) {
    if (a.frequency == b.frequency)
        return a.firstIndex < b.firstIndex;
    return a.frequency > b.frequency;
}

void sortByFrequency(int arr[], int n) {
    // Find the maximum and minimum element in the array
    int maxElem = *max_element(arr, arr + n);
    int minElem = *min_element(arr, arr + n);

    // Use an offset to handle negative values
    int offset = -minElem;
    int range = maxElem - minElem + 1;

    // Frequency array
    int freq[range] = {0};

    // First occurrence array
    int firstIndex[range];
    fill_n(firstIndex, range, -1);

    // Fill frequency and firstIndex arrays
    for (int i = 0; i < n; ++i) {
        int index = arr[i] + offset;
        freq[index]++;
        if (firstIndex[index] == -1) {
```

```
            firstIndex[index] = i;
        }
    }

    // Create an array of Element structures
    Element elements[range];
    int uniqueCount = 0;

    for (int i = 0; i < range; ++i) {
        if (freq[i] > 0) {
            elements[uniqueCount].value = i - offset;
            elements[uniqueCount].frequency = freq[i];
            elements[uniqueCount].firstIndex = firstIndex[i];
            uniqueCount++;
        }
    }

    // Sort the elements array
    sort(elements, elements + uniqueCount, compare);

    // Output the sorted elements based on frequency and first occurrence
    int index = 0;
    for (int i = 0; i < uniqueCount; ++i) {
        for (int j = 0; j < elements[i].frequency; ++j) {
            arr[index++] = elements[i].value;
        }
    }
}
```

	Test	Expected	Got	
✓	\tint arr[] = {-4,1,2,2,-4,9,1,-1}; \tint n = sizeof(arr) / sizeof(arr[0]); \tsortByFrequency(arr, n); \tfor (int i = 0; i < n; i++) \t\tcout << arr[i] << " ";	-4 -4 1 1 2 2 9 -1	-4 -4 1 1 2 2 9 -1	✓
✓	\tint arr[] = {-5,3,8,1,-9,-9}; \tint n = sizeof(arr) / sizeof(arr[0]); \tsortByFrequency(arr, n); \tfor (int i = 0; i < n; i++) \t\tcout << arr[i] << " ";	-9 -9 -5 3 8 1	-9 -9 -5 3 8 1	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **11**

Đúng

Đạt điểm 1,00 trên 1,00

Given a list of points on the 2-D plane (**points[]** with **n** elements) and an integer **k**. Your task in this exercise is to implement the **closestKPoints** function to find K closest points to the given point (**des_point**) and print them by descending order of distances.

Prototype of `closestKPoints`:

```
void closestKPoints(Point points[], int n, Point& des_point, int k);
```

Note: The distance between two points on a plane is the Euclidean distance.

Template:

```
#include <iostream>
#include <string>
#include <cmath>
#include <vector>
#include <algorithm>
```

```
using namespace std;

class Point{
   public:
   int x, y;
   Point(int x = 0, int y = 0){
      this->x = x;
      this->y = y;
   }
   void display(){
      cout << "("<<x<<", "<<y<<")";
   }
};
```

For example:

Test	Result

`Point points[] = {{3, 3},{5, -1},{-2, 4}};`
`int n = sizeof(points)/sizeof(points[0]);`
`int k = 2;`
`Point des_point = {0,2};`
`closestKPoints(points, n, des_point, k);`
	(-2, 4)
	(3, 3)

`Point points[] = {{3, 3},{5, -1},{-2, 4}};`
`int n = sizeof(points)/sizeof(points[0]);`
`int k = 3;`
`Point des_point = {0,2};`
`closestKPoints(points, n, des_point, k);`
	(-2, 4)
	(3, 3)
	(5, -1)

Answer: (penalty regime: 0 %)

Reset answer

```
1
2  // Function to calculate the squared distance between two points
3  double squaredDistance(const Point& p1, const Point& p2) {
4      return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
5  }
6
7  // Function to find and print k closest points
8  void closestKPoints(Point points[], int n, Point& des_point, int k) {
9      // Vector to store pairs of (distance, point)
```

```
    vector<pair<double, Point>> dist_points;

    // Compute distances and store them
    for (int i = 0; i < n; ++i) {
        double dist = squaredDistance(points[i], des_point);
        dist_points.emplace_back(dist, points[i]);
    }

    // Sort the vector based on distances
sort(dist_points.begin(), dist_points.end(), [](const pair<double, Point>& a, const pair<double, Point>& b) {
        return a.first < b.first;
    });

    // Print the k closest points in descending order of distances
    size_t limit = min(static_cast<size_t>(k), dist_points.size());
    for (size_t i = 0; i < limit; ++i) {
        dist_points[i].second.display();
        cout << endl;
    }
    cout << endl;
}
```

	Test	Expected	Got	

✓	Point points[] = {{3, 3},{5, -1},{-2, 4}};
	int n = sizeof(points)/sizeof(points[0]);
	int k = 2;
	Point des_point = {0,2};
	closestKPoints(points, n, des_point, k);
		(-2, 4)
		(3, 3)
			(-2, 4)
			(3, 3)
				✓

✓	Point points[] = {{3, 3},{5, -1},{-2, 4}};
	int n = sizeof(points)/sizeof(points[0]);
	int k = 3;
	Point des_point = {0,2};
	closestKPoints(points, n, des_point, k);
		(-2, 4)
		(3, 3)
		(5, -1)
			(-2, 4)
			(3, 3)
			(5, -1)
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.